Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Моделирование игры в кости

Группа: М8О-210БВ-25

Студентка: Старыгина А.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.11.26

Москва, 2026

Постановка задачи

Разработать программу на языке C для оценки шансов на победу в игре в кости. Каждый из двух игроков бросает две кости K раз; выигрывает игрок с большей итоговой суммой. На вход подаются K, текущий тур r, уже набранные очки a и b, количество экспериментов N и ограничение потоков -t.

Текущий тур ещё предстоит сыграть, поэтому осталось R = K - r + 1 туров. Реализовать последовательное и многопоточное моделирование средствами POSIX Threads, измерить время обеих версий, проверить результат и показать потоки средствами ОС. Исследовать влияние N, R и числа потоков на ускорение и эффективность.

Использованные функции

- pthread_create(), pthread_join() — создание потоков и ожидание их завершения.
- rand_r() — псевдослучайные числа с отдельным состоянием генератора.
- clock_gettime(CLOCK_MONOTONIC, ...) — измерение длительности вычислений.
- strtol(), strcmp(), printf(), sleep() — аргументы, вывод и пауза для просмотра.

Описание алгоритма

В каждом из N экспериментов копируются текущие очки и моделируются оставшиеся туры. Каждому игроку за тур добавляются значения двух костей от 1 до 6. Затем увеличивается счётчик побед первого, побед второго или ничьих. Оценки вероятностей равны C1/N, C2/N и Cd/N; их сумма равна единице.

Начальное состояние rand_r зависит от номера эксперимента: (i + 1) * 2654435761u. Остаток диапазона, не кратный шести, отбрасывается. Один опыт использует одинаковые значения при любом числе потоков; повторные запуски воспроизводят ту же выборку.

Номера опытов делятся на участки [N*i/p, N*(i+1)/p). Каждый поток ведёт собственные счётчики; после pthread_join основной поток объединяет их. Используется p = min(лимит, N), включая основной поток, поэтому создаётся p-1 новых. Мьютекс не требуется. Работа составляет O(N(R+1)), служебная память — O(p).

Метрики производительности

Ускорение S показывает отношение последовательного времени Tseq к параллельному Tp. Число p — фактическое количество участников вычислений, включая основной поток.

$$S = \frac{Tseq}{Tp}$$

Эффективность E — ускорение на одного участника в процентах. Она характеризует отдачу от распараллеливания и не равна загрузке процессора.

$$E = \frac{S}{p} \cdot 100\,\%$$

Код программы

main.c

```
#include "game.h"

#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

void error(const char *text) {
    fprintf(stderr, "%s", text);
    exit(1);
}

int read_number(const char *text) {
    char *end;
    errno = 0;
    long value = strtol(text, &end, 10);
    if (errno != 0 || end == text || *end != '\0' || value < 0 || value > INT_MAX) {
        error("Ожидалось неотрицательное целое число типа int.\n");
    }
    return (int)value;
}

double current_time_ms(void) {
    struct timespec time;
    clock_gettime(CLOCK_MONOTONIC, &time);
    return time.tv_sec * 1000.0 + time.tv_nsec / 1000000.0;
}

int main(int argc, char *argv[]) {
    if ((argc != 13 && argc != 14) || strcmp(argv[1], "-k") != 0 ||
        strcmp(argv[3], "-r") != 0 || strcmp(argv[5], "-a") != 0 ||
        strcmp(argv[7], "-b") != 0 || strcmp(argv[9], "-n") != 0 ||
        strcmp(argv[11], "-t") != 0 || (argc == 14 && strcmp(argv[13], "-s") != 0)) {
        error("Использование: ./main -k K -r ТУР -a ОЧКИ1 -b ОЧКИ2 "
              "-n ОПЫТЫ -t ПОТОКИ [-s]\n");
    }
    Game game;
    game.k = read_number(argv[2]);
    game.round = read_number(argv[4]);
    game.score1 = read_number(argv[6]);
    game.score2 = read_number(argv[8]);
    game.experiments = read_number(argv[10]);
    int limit = read_number(argv[12]);
    int inspect = argc == 14;
    if (game.k == 0 || game.k == INT_MAX || game.round == 0 || game.round > game.k + 1 ||
        game.experiments == 0 || limit == 0 || limit > 1024) {
        error("Нужно: K от 1 до INT_MAX-1, тур от 1 до K+1, "
              "опыты > 0, потоки от 1 до 1024.\n");
    }
    int threads = limit;
    if (threads > game.experiments) threads = game.experiments;

    Result sequential, parallel;
    double start = current_time_ms();
    simulate_sequential(&game, &sequential);
    double sequential_ms = current_time_ms() - start;
    start = current_time_ms();
    simulate_parallel(&game, &parallel, threads, inspect);
    double parallel_ms = current_time_ms() - start;
    int ok = results_equal(&sequential, &parallel);

    printf("Всего туров: %d\nТекущий тур: %d\nОсталось туров: %d\n",
           game.k, game.round, game.k - game.round + 1);
```

```
    printf("Очки игроков: %d %d\nЭкспериментов: %d\n",
           game.score1, game.score2, game.experiments);
    printf("Лимит потоков: %d\nИспользовано потоков: %d\n", limit, threads);
    printf("Побед первого: %lld\nПобед второго: %lld\nНичьих: %lld\n",
           parallel.wins1, parallel.wins2, parallel.draws);
    printf("Шанс победы первого, %%: %.4f\n", 100.0 * parallel.wins1 / game.experiments);
    printf("Шанс победы второго, %%: %.4f\n", 100.0 * parallel.wins2 / game.experiments);
    printf("Шанс ничьей, %%: %.4f\n", 100.0 * parallel.draws / game.experiments);
    if (!inspect) {
        double speedup = parallel_ms > 0 ? sequential_ms / parallel_ms : 0;
        printf("Последовательно, мс: %.3f\nПараллельно, мс: %.3f\n",
               sequential_ms, parallel_ms);
        printf("Ускорение: %.3f\nЭффективность, %%: %.3f\n",
               speedup, speedup / threads * 100);
    }
    printf("Проверка: %s\n", ok ? "OK" : "ОШИБКА");
    return ok ? 0 : 1;
}
```

game.h

```
#ifndef GAME_H
#define GAME_H

typedef struct {
    int k;
    int round;
    int score1, score2;
    int experiments;
} Game;

typedef struct {
    long long wins1, wins2, draws;
} Result;

void error(const char *text);
void simulate_part(const Game *game, long long begin, long long end, Result *result);
void simulate_sequential(const Game *game, Result *result);
void simulate_parallel(const Game *game, Result *result, int threads, int inspect);
int results_equal(const Result *a, const Result *b);

#endif
```

game.c

```
#include "game.h"

#include <stdlib.h>

int throw_die(unsigned int *seed) {
    int value;
    int limit = RAND_MAX - RAND_MAX % 6;
    do {
        value = rand_r(seed);
    } while (value >= limit);
    return value % 6 + 1;
}

void simulate_part(const Game *game, long long begin, long long end, Result *result) {
    Result count = {0, 0, 0};
    for (long long i = begin; i < end; i++) {
        unsigned int seed = ((unsigned int)i + 1) * 2654435761u;
        long long score1 = game->score1;
        long long score2 = game->score2;
        for (int round = game->round; round <= game->k; round++) {
            score1 += throw_die(&seed) + throw_die(&seed);
            score2 += throw_die(&seed) + throw_die(&seed);
        }
        if (score1 > score2) count.wins1++;
        else if (score2 > score1) count.wins2++;
        else count.draws++;
    }
```

```
    *result = count;
}

void simulate_sequential(const Game *game, Result *result) {
    simulate_part(game, 0, game->experiments, result);
}

int results_equal(const Result *a, const Result *b) {
    return a->wins1 == b->wins1 && a->wins2 == b->wins2 && a->draws == b->draws;
}
```

parallel.c

```
#include "game.h"

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

typedef struct {
    const Game *game;
    long long begin, end;
    Result result;
    int inspect;
} ThreadData;

void *worker(void *argument) {
    ThreadData *data = argument;
    if (data->inspect) sleep(6);
    simulate_part(data->game, data->begin, data->end, &data->result);
    return NULL;
}

void simulate_parallel(const Game *game, Result *result, int threads, int inspect) {
    if (threads == 1 && !inspect) {
        simulate_sequential(game, result);
        return;
    }
    pthread_t ids[threads];
    ThreadData data[threads];
    for (int i = 0; i < threads; i++) {
        data[i].game = game;
        data[i].begin = (long long)game->experiments * i / threads;
        data[i].end = (long long)game->experiments * (i + 1) / threads;
        data[i].inspect = inspect;
        if (i > 0 && pthread_create(&ids[i], NULL, worker, &data[i]) != 0) {
            error("Не удалось создать потоки. Уменьшите -t.\n");
        }
    }
    if (inspect) {
        printf("PID: %ld. Потоки созданы, пауза 6 секунд.\n", (long)getpid());
        fflush(stdout);
    }
    worker(&data[0]);
    for (int i = 1; i < threads; i++) {
        if (pthread_join(ids[i], NULL) != 0) error("Ошибка ожидания потока.\n");
    }
    result->wins1 = 0;
    result->wins2 = 0;
    result->draws = 0;
    for (int i = 0; i < threads; i++) {
        result->wins1 += data[i].result.wins1;
        result->wins2 += data[i].result.wins2;
        result->draws += data[i].result.draws;
    }
}
```

Протокол работы программы

Makefile

```
CC = cc
CFLAGS = -std=c11 -O2 -Wall -Wextra -Wpedantic -D_POSIX_C_SOURCE=200809L -pthread

main: main.c game.c parallel.c game.h
	$(CC) $(CFLAGS) main.c game.c parallel.c -o main

clean:
	rm -f main

THREADS ?= 14

stats: main
	@./main -k 20 -r 11 -a 70 -b 75 -n 10000 -t $(THREADS) -s & \
	pid=$$!; sleep 2; ps -M -p $$pid; wait $$pid

.PHONY: clean stats
```

Сборка и запуск для K = 20, текущего тура 11, очков 70 и 75, N = 200000 и лимита 14 потоков:

```
make
./main -k 20 -r 11 -a 70 -b 75 -n 200000 -t 14
```

Один из выполненных запусков:

```
Всего туров: 20
Текущий тур: 11
Осталось туров: 10
Очки игроков: 70 75
Экспериментов: 200000
Лимит потоков: 14
Использовано потоков: 14
Побед первого: 61329
Побед второго: 132104
Ничьих: 6567
Шанс победы первого, %: 30.6645
Шанс победы второго, %: 66.0520
Шанс ничьей, %: 3.2835
Последовательно, мс: 35.170
Параллельно, мс: 4.092
Ускорение: 8.595
Эффективность, %: 61.392
Проверка: OK
```

Ключ -k задаёт общее число туров, -r — первый ещё не сыгранный тур, -a и -b — текущие очки, -n — число опытов, -t — лимит участников. При r = 1 игра ещё не началась; при r = K+1 осталось ноль туров и сравниваются введённые суммы. Аргументы передаются в указанном порядке.

Просмотр потоков средствами ОС

На macOS команда ниже запускает режим -s: каждый участник ждёт шесть секунд, а ps -M показывает потоки. В проверочном запуске PID 96294 наблюдалось 14 потоков: основной и 13 созданных. Время в этом режиме не выводится.

```
make stats THREADS=14
```

На Linux список потоков можно получить в том же терминале:

```
./main -k 20 -r 11 -a 70 -b 75 -n 10000 -t 14 -s &
pid=$!; sleep 2; ps -L -p "$pid"; wait "$pid"
```

Проверка моделирования

При одном оставшемся туре и равных очках перебор 1296 сочетаний четырёх костей даёт шансы победы каждого 44,3673 % и ничьей 11,2654 %. Для N = 1000000 получены 44,3658 %, 44,3554 % и 11,2788 %.

Результаты экспериментов

Замеры выполнены 04.11.2026 на Apple M4 Pro с 14 логическими ядрами и 48 ГиБ памяти, macOS 26.6.2, Apple Clang 17, оптимизация -O2. Для каждого p выполнено три запуска. Четыре значения 1, 2, 4, 8 меньше числа ядер, 14 равно ему, а 64, 128 и 1024 значительно больше.

Время исполнения — среднее Tp. Общий Tseq для каждого набора усреднён по всем 24 последовательным замерам этого набора. Время включает генерацию бросков и подсчёт; для Tp также создание потоков, ожидание и объединение. Аргументы, вывод и проверка не учитываются. Повторные запуски нужны для замеров, а число различных моделируемых исходов остаётся N.

Набор 1 N = 200 000

K = 20, r = 11, a = 70, b = 75; осталось 10 туров. Tseq = 33,503 мс.

Число потоков	Время исполнения, мс	Ускорение	Эффективность, %
1	33,944	0,987	98,70
2	16,373	2,046	102,31
4	9,274	3,612	90,31
8	5,335	6,280	78,50
14	4,627	7,240	51,72
64	3,658	9,159	14,31
128	3,505	9,559	7,47
1024	11,582	2,893	0,28

Набор 2 N = 2 000 000

K = 20, r = 11, a = 70, b = 75; осталось 10 туров. Tseq = 322,707 мс.

Число потоков	Время исполнения, мс	Ускорение	Эффективность, %
1	321,031	1,005	100,52
2	163,853	1,969	98,47
4	93,326	3,458	86,45
8	47,747	6,759	84,48
14	38,669	8,345	59,61
64	33,678	9,582	14,97
128	33,252	9,705	7,58
1024	34,706	9,298	0,91

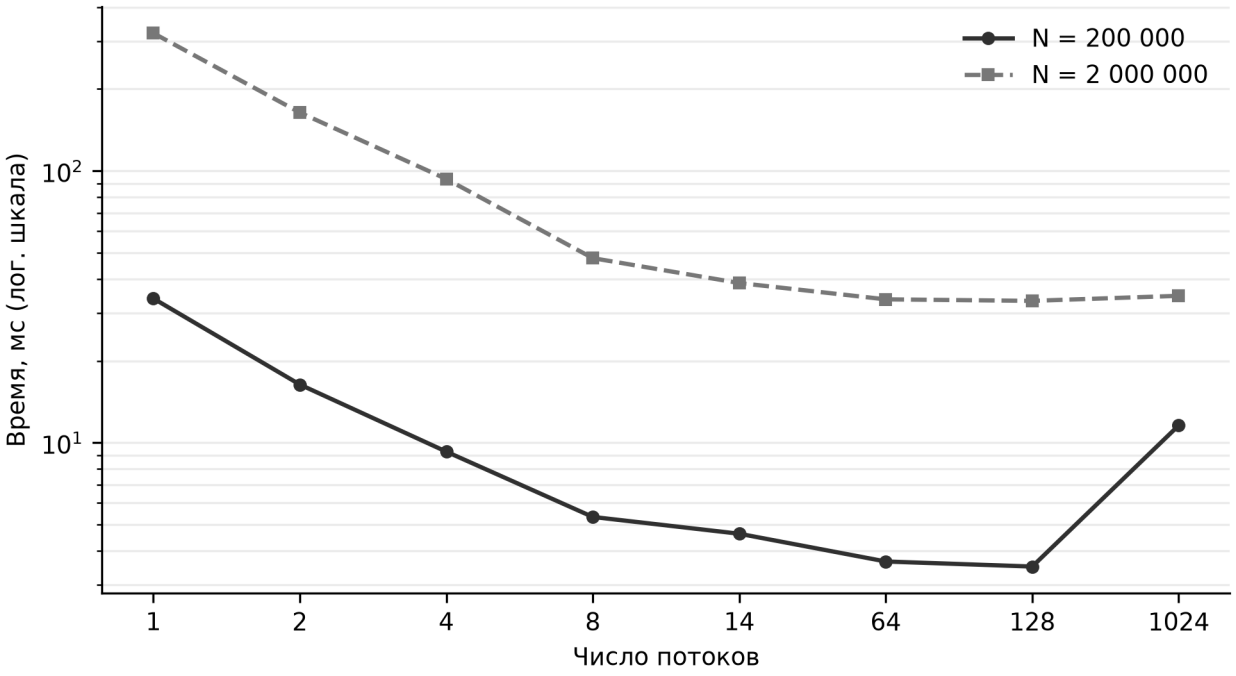


Рисунок 1 — Время выполнения при разном числе потоков

Влияние количества экспериментов

При K = 20, r = 11, a = 70, b = 75 и p = 14 получены следующие средние значения трёх запусков. Для каждой строки S рассчитано по её собственным средним Tseq и Tp.

Опыты N	Tseq, мс	Tp, мс	Ускорение
100	0,016	0,099	0,166
1 000	0,161	0,116	1,385
10 000	1,614	0,312	5,169
100 000	16,467	2,102	7,835
200 000	33,816	4,627	7,308

Влияние оставшихся туров

При N = 200000, a = 70, b = 75 и p = 14 изменялось число оставшихся туров. Для R = 1 использованы K = 20, r = 20; для R = 90 — K = 100, r = 11. Строка R = 10 взята из основного набора.

Туров R	Tseq, мс	Tp, мс	Ускорение
1	1,861	0,352	5,291
10	33,816	4,627	7,308
90	325,736	36,763	8,860

Увеличение N одновременно повышает объём работы и уточняет оценку вероятности. Увеличение R добавляет броски внутри каждого опыта. При N = 100 создание потоков дороже вычислений: ускорение меньше единицы.

Анализ результатов

Для N = 200 000 минимальное среднее время 3,505 мс получено при 128 потоках: S = 9,559 и E = 7,47 %. Для N = 2 000 000 минимум также наблюдался при 128 потоках: 33,252 мс, S = 9,705 и E = 7,58 %. Это лучшие значения данной серии из трёх запусков, а не гарантия оптимальности этого числа потоков.

Результаты 64 и 128 потоков различаются примерно на 4 % для малого набора и на 1 % для большого. Различия могут быть связаны с планированием независимых опытов и фоновой нагрузкой. По такой короткой серии нельзя утверждать, что 128 потоков устойчиво лучше 64. При этом эффективность при 14 потоках заметно выше, поскольку сопоставимое ускорение достигается меньшим числом участников.

При 1024 потоках малый набор замедлился до 11,582 мс, а эффективность снизилась до 0,28 %. Для большого набора время составило 34,706 мс: полезной работы на поток больше, поэтому дополнительные расходы менее заметны. Число аппаратных исполнителей не растёт вместе с числом программных потоков, а создание и планирование требуют времени.

На маленьком N = 100 параллельный запуск медленнее последовательного, поскольку всего моделируется 100 завершений игры. При увеличении N и числа оставшихся туров доля служебных расходов уменьшается. Основная часть работы — генерация бросков в независимых опытах; каждый участник имеет собственное состояние генератора и локальные счётчики.

В отдельных строках эффективность немного превышает 100 %. В частности, для малого набора при двух потоках E = 102,31 %. Это оценка по измеренным временам и общей усреднённой базе Tseq: на неё влияют разброс, частота CPU и прогрев кода. Она не означает загрузку процессора более 100 %. Более длинные серии нужны для устойчивых выводов о небольших различиях.

При одинаковых параметрах оба алгоритма дали одни и те же целочисленные счётчики для всех проверенных p. Например, для N = 200000 получены 61329 побед первого, 132104 победы второго и 6567 ничьих. Для N = 2000000 — 612093, 1321879 и 66028. Второй игрок чаще выигрывает благодаря преимуществу в пять очков, но исход остаётся случайным. При равных начальных очках контрольный пример согласуется с точными вероятностями.

Вывод

Реализованы последовательная и параллельная оценки шансов игроков методом Монте-Карло на языке C. Работа разделена по номерам опытов, ограничение -t учитывает основной поток, а ожидание выполнено через pthread_join. Собственные состояния генератора и счётчики позволили обойтись без мьютекса и точно сравнить результаты версий.

Распараллеливание ускорило достаточно большие задачи. Лучшее измеренное ускорение составило 9,705 при 128 потоках; близкое время получено при 64. При чрезмерном числе участников эффективность падает, а на очень малом объёме данных создание потоков обходится дороже моделирования. Параметры N и R нужно учитывать вместе с количеством потоков.